

Planar Tensor Networks, Braiding, and Interface Invariants

June 25, 2026

Abstract

This note records the design consequences of supporting non-Abelian symmetries, fermionic signs, and general braiding in a tensor-network library. The main point is simple: once braiding or nontrivial recoupling is present, a tensor expression is not determined by the set of labels on its legs. It is determined by an ordered and possibly planar diagram. A robust interface therefore needs explicit objects for spaces, dual spaces, ordered legs, fusion trees, planar network geometry, and contraction plans. Mutable labels may remain as display metadata or as local notation inside a contraction expression, but they cannot be the canonical identity of tensor legs.

Contents

1	Purpose	2
2	Minimal Categorical Model	2
2.1	Strict and Symmetric Cases	3
2.2	Non-Abelian and Braided Cases	3
3	What “Planar” Means	3
3.1	Local Order vs Global Planarity	4
3.2	Why a Tensor Alone Cannot Know Whether a Braid Is Needed	4
4	Three-Layer Design	4
4.1	Layer 1: Tensor Objects	4
4.2	Layer 2: Planar Network Objects	5
4.3	Layer 3: Contraction Plans	5
5	Planar Invariants	5
5.1	Boundary Invariant	6
5.2	Dual Compatibility Invariant	6
5.3	Space Identity Invariant	6
5.4	Cyclic Order Invariant	6
5.5	No Implicit Crossing Invariant	6
5.6	Fusion-Tree Invariant	6
5.7	Braid Accounting Invariant	7
5.8	Trace and Twist Invariant	7
5.9	Decomposition Invariant	7
5.10	Metadata Is Not Semantics	7
6	Operation Taxonomy	7
6.1	Reorder Operations	8

7	TensorKit.jl as a Useful Reference Point	8
7.1	Fusion Trees	8
7.2	TensorKit’s Hierarchy of Reordering Operations	8
7.3	@tensor, @planar, and @plansor	9
8	A Possible uni20 Interface Shape	9
8.1	Spaces and Legs	9
8.2	Contraction by Explicit Legs	9
8.3	Network-Level Planar Contraction	9
8.4	Einsum-Like Syntax as a Front End	10
9	Diagnostics	10
10	Testing the Invariants	10
10.1	Dense Space Identity Tests	10
10.2	SVD Bond Creation Tests	11
10.3	Relabel Non-Effect Tests	11
10.4	Braiding Tests	11
10.5	Fusion Recoupling Tests	11
11	Consequences for Cytnx Compatibility	11
12	Conclusion	12

1 Purpose

The immediate motivation is the failure mode of Cytnx-style tensor labels. In Cytnx, labels are persistent mutable strings attached to tensor legs, and many contractions use those labels to decide which legs to contract. This can be convenient for dense tensors, but it is not a sound foundation for tensor categories with nontrivial associators, fermionic signs, or braiding.

The target design here is closer to what a future uni20 interface should enforce. It is not a proposal to reproduce the Cytnx C++ object model. In particular:

- dense arrays should be ordinary array objects, not the central tensor-network abstraction;
- tensor-network correctness should be expressed in terms of spaces, dual spaces, ordered legs, morphisms, and diagrams;
- high-level notation may look like einsum, but it must compile to a checked contraction plan;
- labels are useful for display and diagnostics, but they should not be the authority on mathematical identity.

2 Minimal Categorical Model

A tensor-network library with symmetries is not merely an ndarray library with labels. It is manipulating morphisms in a monoidal category $\mathcal{C}$. The objects of $\mathcal{C}$ are spaces, and tensors are maps between tensor products of spaces.

For a tensor A , write

$$A : X_1 \otimes X_2 \otimes \cdots \otimes X_m \longrightarrow Y_1 \otimes Y_2 \otimes \cdots \otimes Y_n.$$

Equivalently,

$$A \in \text{Hom}_{\mathcal{C}}(X_1 \otimes \cdots \otimes X_m, Y_1 \otimes \cdots \otimes Y_n).$$

The distinction between domain and codomain is not optional. A contraction pairs a space with a compatible dual space. For ordinary finite-dimensional vector spaces, this distinction can be flattened away by choosing bases, but tensor-network code with symmetries should not erase it.

2.1 Strict and Symmetric Cases

Dense tensors over ordinary vector spaces live in a symmetric monoidal category. In this setting:

- associators can usually be treated as identities;
- swapping two factors is represented by an ordinary permutation;
- there is no sign, R -matrix, or recoupling coefficient;
- many illegal-looking relabellings accidentally produce the intended dense numerical answer.

This is the misleading special case. It trains users to believe that “if the labels are right, the order does not matter.” That is not a valid rule in the general setting.

2.2 Non-Abelian and Braided Cases

For non-Abelian symmetries, a basis for a multi-leg tensor depends on the fusion tree. For example,

$$((a \otimes b) \otimes c) \rightarrow d$$

and

$$(a \otimes (b \otimes c)) \rightarrow d$$

are related by an F -move. They are not the same basis with different leg names.

For braided tensor categories, exchanging two objects applies an R -move:

$$R_{a,b} : a \otimes b \rightarrow b \otimes a.$$

For fermionic tensors this may reduce to parity signs, but the same interface lesson applies: a swap is a mathematical operation, not a metadata edit.

3 What “Planar” Means

In this note, a planar tensor network is not just a graph. It is a graph with enough embedding data to say which tensor legs are adjacent and whether a proposed contraction crosses another strand.

At minimum, a planar network needs:

- vertices, each representing a tensor or elementary morphism;
- half-edges attached to ordered tensor legs;
- internal edges pairing compatible half-edges;
- external boundary legs, in a specified boundary order;
- a cyclic order of half-edges around every vertex;
- enough embedding information to decide whether adding or reordering an edge introduces a crossing;
- explicit crossing or braiding nodes when crossings are intended.

For a dense bosonic tensor network, most of this can be ignored. For a braided category, it is part of the expression.

3.1 Local Order vs Global Planarity

A single tensor can store a local ordered list of legs:

$$[\ell_0, \ell_1, \dots, \ell_{r-1}].$$

That is necessary, but not sufficient. It says the order around that one tensor. It does not fully determine the embedding of a multi-tensor network.

The global network must additionally know how tensor vertices are placed relative to each other, or at least an equivalent combinatorial embedding. Without this information, the library cannot distinguish between these two cases:

two non-crossing contractions vs. the same pairings with one crossing.

In a symmetric category those two diagrams are equivalent after an ordinary permutation. In a braided category they differ by R -moves.

3.2 Why a Tensor Alone Cannot Know Whether a Braid Is Needed

Whether a braid is needed is not a property of one tensor in isolation. It depends on the larger expression.

For example, suppose tensor A has outgoing boundary order

$$[a, b]$$

and tensor B has incoming boundary order

$$[a^*, b^*].$$

Contracting a with a^* and b with b^* may be planar in one embedding. Contracting a with b^* and b with a^* requires a crossing unless the tensors are embedded differently or explicit braids have been inserted.

The individual tensors only know their local boundary orders. The network or contraction planner must decide whether the pairings are planar.

4 Three-Layer Design

A robust implementation should separate three layers.

4.1 Layer 1: Tensor Objects

A tensor object stores local categorical data:

- an ordered domain list;
- an ordered codomain list;
- the corresponding spaces and dual spaces;
- optional display labels attached to spaces or legs;
- symmetry-sector data and degeneracy data;
- fusion-tree data, if the category is non-Abelian;
- reduced numerical blocks.

This layer can answer questions such as:

- What is the i -th leg?
- Is this leg a **Space** or **CoSpace**?
- What sectors live on this leg?
- What fusion tree defines the basis?
- Does this tensor have the same boundary type as another tensor?

It cannot, by itself, decide all global planarity questions.

4.2 Layer 2: Planar Network Objects

A network object stores a diagram:

- tensor vertices;
- internal pairings;
- external boundary order;
- cyclic orders around vertices;
- explicit braid/crossing vertices if present;
- optional geometric coordinates for display, not as the authority on topology.

The network object is the right place to ask:

- Is this proposed contraction planar?
- Would contracting these two legs cross an existing edge?
- What is the boundary order after contraction?
- Is an explicit braid required?
- Can this diagram be simplified by isotopy?

4.3 Layer 3: Contraction Plans

A contraction plan is an executable sequence of local operations. It may be produced from a network after checking the diagram.

The plan records:

- which local contractions are performed;
- which tensor products are formed;
- which transposes are merely cyclic reorders;
- which F -moves are inserted;
- which R -moves are inserted;
- which traces include twists or supertraces;
- the expected final boundary order and spaces.

This is the layer that can optimize. The tensor object should not silently optimize away categorical distinctions.

5 Planar Invariants

The following invariants are the core of the design. They are the rules that should remain true after every operation.

5.1 Boundary Invariant

Every tensor and every network has a well-defined ordered boundary. For a tensor this is the ordered domain and codomain. For a network it is the ordered list of uncontracted external legs.

An operation must specify the boundary it produces. It is not enough to produce numerically shaped data.

5.2 Dual Compatibility Invariant

A contraction can only pair compatible dual spaces. In the simplest case this means V with V^* . With symmetries it also means compatible charge sectors, degeneracies, and orientations.

Equal total dimension is not sufficient. Equal label string is not sufficient.

5.3 Space Identity Invariant

Two spaces may have the same dimension and the same charge sectors but still represent different positions in a tensor network. For example, two MPS bond spaces at different cuts may both have dimension 32. They should not be interchangeable unless the library has explicit evidence that they are the same space.

This is where labels are useful if they label spaces, not tensor-leg positions. A finite MPS can label its bond spaces

$$b_0, b_1, \dots, b_L,$$

and then contractions over the wrong bond fail even in the dense case.

5.4 Cyclic Order Invariant

At every tensor vertex, the cyclic or linear order of legs is structural data. Reordering legs changes the tensor expression unless the operation is explicitly known to be valid.

Dense tensors may implement this as a cheap axis permutation. Braided or non-Abelian tensors cannot treat it as mere bookkeeping.

5.5 No Implicit Crossing Invariant

No operation may introduce a crossing silently.

If a proposed contraction is non-planar, the operation should fail with a diagnostic such as:

This contraction is not planar in the current diagram. Insert an explicit braid, or use a contraction planner that is allowed to insert braids.

The important point is that the user learns what mathematical operation is missing.

5.6 Fusion-Tree Invariant

For non-Abelian categories, every tensor basis is defined relative to a fusion tree or an equivalent canonical representation.

If an operation changes the parenthesization of a tensor product, it must either:

- prove that no recoupling is needed;
- apply the appropriate F -move;
- or fail.

Changing labels must never be able to bypass this.

5.7 Braid Accounting Invariant

If two strands are exchanged in a braided category, the corresponding R -move must appear in the tensor expression, either as an explicit operation or as an explicit node in the contraction plan.

The library should distinguish:

- a permutation in a symmetric category;
- a fermionic swap with parity signs;
- a genuine anyonic braid with an R -matrix;
- a display-only reorder.

5.8 Trace and Twist Invariant

Closing a line into a trace is not always the same operation as a matrix trace. In fermionic or braided settings, trace-like operations may require twists, signs, or categorical traces.

The interface should therefore distinguish ordinary dense traces from categorical traces. A planner should not silently replace one with the other.

5.9 Decomposition Invariant

SVD, QR, LQ, polar decomposition, and related operations are not just matrix factorizations. In a tensor-network context they create new intermediate spaces.

For an SVD of

$$A : X \rightarrow Y,$$

the decomposition

$$A = USV$$

creates an intermediate space B . The singular-value object S is an endomorphism of B :

$$S : B \rightarrow B.$$

Thus S naturally has the same space label on its two sides. A global rule that tensor labels must be unique is incompatible with this mathematical structure.

5.10 Metadata Is Not Semantics

Display labels, names, colors, and diagram coordinates help humans. They are not the authority on which operations are legal.

Changing a display label should never alter contraction legality. Changing a space identity or applying a braid should be a separate, checked operation.

6 Operation Taxonomy

The interface should split operations that dense tensor libraries often conflate.

Operation	Dense interpretation	General categorical interpretation
<code>relabel</code>	rename an axis	display metadata only
<code>permute</code>	reorder axes	may require braid or recoupling
<code>transpose</code>	swap matrix axes	cyclic/domain-codomain move, if valid
<code>reshape</code>	reinterpret shape	usually not meaningful for spaces
<code>contract</code>	sum over axes	compose compatible morphisms
<code>trace</code>	sum diagonal	categorical trace may need twist/sign
<code>svd</code>	matrix SVD	factorization creating a new bond space
<code>get_block</code>	array slice	reduced block in a chosen fusion basis

6.1 Reorder Operations

At least four operations should be distinct:

relabel Change display names only.

repartition Move the domain/codomain boundary without changing the cyclic order, when categorically valid.

transpose Perform a cyclic reorder that introduces no crossing.

braid Exchange adjacent strands using an R -move.

Dense libraries collapse these into one or two functions. That is exactly what should be avoided in a category-aware tensor-network interface.

7 TensorKit.jl as a Useful Reference Point

TensorKit.jl provides a useful example of this separation. I am not claiming uni20 should copy its interface exactly, but its design validates the main point of this note: once braiding matters, ordinary implicit tensor notation is not enough.

The TensorKit documentation describes tensors as **TensorMap** objects, i.e. maps from a domain to a codomain. It also has explicit fusion trees, with uncoupled sectors, coupled sectors, duality flags, inner fusion channels, and vertex labels. This is precisely the kind of information that a Cytnx-style leg label cannot encode.

7.1 Fusion Trees

TensorKit's fusion-tree data stores the basis structure of a symmetric tensor. The important lesson is that block sparsity is not just a dictionary keyed by quantum numbers. It is block data together with a choice of fusion basis.

For uni20, this suggests:

- tensor blocks should be reduced data;
- fusion paths should be explicit or canonical;
- recoupling should be an operation on the basis, not an accidental consequence of permuting axes.

7.2 TensorKit's Hierarchy of Reordering Operations

TensorKit distinguishes several index manipulations:

braid The most general operation. It can represent crossings and needs information about over/under order.

permute A restricted operation for sector types with symmetric braiding, where over/under crossings are equivalent.

transpose Cyclic permutations that do not introduce crossings.

repartition Moving the boundary between domain and codomain without arbitrary reordering.

This is a useful vocabulary. It explicitly prevents the common dense tensor mistake of treating every leg reorder as an axis permutation.

7.3 @tensor, @planar, and @plansor

TensorKit's contraction macros make the planar issue concrete.

For bosonic or symmetric cases, ordinary tensor notation can be enough. TensorKit's `@tensor` macro handles implicit contractions in that setting.

For anyonic or braided cases, the same implicit notation is not sufficient. TensorKit provides a `@planar` macro where the expression must describe a planar diagram. Crossings are not silently inferred; they are represented explicitly by a reserved braiding tensor such as τ , with τ' for the inverse crossing.

TensorKit also provides `@plansor`, which dispatches to the ordinary tensor machinery in bosonic cases and to planar machinery when the category requires it.

The design lesson is strong: there are two different languages.

- A non-planar indexed expression language, acceptable only when the category is symmetric enough.
- A planar diagram language, required when crossings have mathematical content.

8 A Possible uni20 Interface Shape

The following sketches are illustrative, not final API.

8.1 Spaces and Legs

```
phys = Space("SpinHalf", symmetry=SU2, sectors=[j(1,2)])
b0    = Space("bond-0", symmetry=SU2, sectors=[j(0)])
b1    = Space("bond-1", symmetry=SU2, sectors=[j(0), j(1)])

A = Tensor(
    codomain=[b1],
    domain=[b0, phys],
    fusion_tree=...
)
```

The key point is that the tensor is a morphism. Its boundary is ordered. Labels such as "bond-0" are attached to spaces for diagnostics and compatibility checks.

8.2 Contraction by Explicit Legs

```
C = contract(A, B, pairs=[(A.codomain[0], B.domain[0])])
```

This operation checks:

- dual compatibility;
- space identity or allowed isomorphism;
- sector compatibility;
- fusion-tree compatibility;
- planarity in the current network context.

8.3 Network-Level Planar Contraction

```
N = PlanarNetwork()
N.add_tensor("A", A)
N.add_tensor("B", B)
```

```
N.connect(("A", 0), ("B", 1))
```

```
plan = N.compile(boundary=[...])
C = plan.evaluate()
```

If a crossing is needed, compilation fails unless it is present in the diagram or explicitly authorized:

```
N.braid(("A", 1), ("B", 0), over=True)
```

8.4 Einsum-Like Syntax as a Front End

An einsum-like expression can still exist:

```
C = einsum("lir,rjs->lijs", A, B)
```

but this should be a notation parser. It should produce a contraction plan and then check that the local labels agree with the tensors' actual spaces. The labels in the expression should not mutate the tensors.

9 Diagnostics

The error messages matter. The library should tell users which categorical condition failed.

Examples:

Cannot contract leg 2 of A with leg 0 of B : spaces have equal dimension but different identities, `bond-4` vs `bond-5`.

Cannot contract these legs in the current planar diagram: the pairing would cross an existing strand. Insert an explicit braid or use a planner with braiding enabled.

Cannot reorder legs by metadata relabel. This reorder changes the fusion tree. Apply an explicit F -move or call `recouple`.

Cannot use ordinary trace on this fermionic/braided tensor. Use the categorical trace operation and specify the required twist convention.

These diagnostics are not just niceties. They are part of the correctness model. A useful tensor-network library should make the wrong operation fail at the point where the wrong assumption is made.

10 Testing the Invariants

The invariants above should be testable.

10.1 Dense Space Identity Tests

Even with no symmetry, create two spaces with the same dimension but different identities:

$$V = \mathbb{C}^{32}, \quad W = \mathbb{C}^{32}.$$

Contraction of V with W^* should fail unless an explicit isomorphism is supplied. This catches many MPS bond-index mistakes that dense tensor libraries silently allow.

10.2 SVD Bond Creation Tests

SVD should create a new bond space B . The singular-value object should be an endomorphism $B \rightarrow B$. Tests should verify that the two sides of S carry the same space identity and that USV reconstructs the original tensor with the expected boundary.

10.3 Relabel Non-Effect Tests

Changing display labels must not change which contractions are legal. If a contraction fails before relabelling, and no space identity, duality, braid, or recoupling operation has been applied, it should still fail after relabelling.

10.4 Braiding Tests

In a category with nontrivial R -symbols, compare:

- a planar contraction with no crossing;
- the same pairings with one explicit braid;
- a rejected attempt to produce the second result by relabelling or ordinary permutation.

The first two may both be legal, but they should generally produce different morphisms. The third should fail.

10.5 Fusion Recoupling Tests

Construct tensors in two different fusion bases,

$$((ab)c)d \quad \text{and} \quad (a(bc))d.$$

Verify that direct comparison or contraction fails unless an explicit F -move or canonical recoupling is applied.

11 Consequences for Cytnx Compatibility

A Cytnx-compatible migration layer can preserve some surface spelling, but it should not preserve unsafe semantics.

- Persistent labels can be read as display labels.
- Label-based contractions can be parsed into explicit leg contractions, then checked.
- Duplicate labels should not be globally forbidden, because endomorphisms naturally have the same space on both sides.
- Relabelling should not make a previously illegal contraction legal.
- Rowrank-style positional decomposition should be replaced by explicit left/right leg groups.
- Block access should be expert-level and described as reduced basis access, not ordinary ndarray slicing.

This means a fully bug-compatible shim is neither possible nor desirable. The useful compatibility target is:

Old scripts either run with the intended semantics, or fail with a diagnostic that tells the user what mathematical structure was missing.

12 Conclusion

Planarity is not decoration. In a braided or non-Abelian tensor category, the diagram is part of the expression. A tensor object knows its local ordered boundary, but the network or planner must know the global planar embedding. Any interface that lets users repair contractions by mutating labels has already lost the essential correctness property.

The practical design rule is:

Use labels for notation and diagnostics. Use spaces, duals, ordered legs, fusion trees, and planar diagrams for semantics.

References

- [1] TensorKit.jl documentation. <https://quantumkithub.github.io/TensorKit.jl/stable/>
- [2] TensorKit.jl manual: fusion trees. <https://quantumkithub.github.io/TensorKit.jl/stable/man/fusiontrees/>
- [3] TensorKit.jl manual: index manipulations. <https://quantumkithub.github.io/TensorKit.jl/stable/man/indexmanipulations/>
- [4] TensorKit.jl manual: contractions, including @tensor, @planar, and @plansor. <https://quantumkithub.github.io/TensorKit.jl/stable/man/contractions/>